



Licenciatura em Engenharia Informática e de Computadores

Gestão de Mercados Financeiros

Trabalho realizado por:

Nome: Dinis Melo N° 51500

Nome: Gabriel Subutzki N° 50142

Nome: David António N° 50495

Docentes: Ana Beire, João Vitorino, Matilde Pato e Nuno Datia

Sistemas de Informação

2025 / 2026 Verão

06 de junho de 2026

<< Esta página foi intencionalmente deixada em branco >>

Resumo

O presente projeto incide sobre o desenvolvimento de mecanismos de integridade e lógica de negócio numa base de dados relacional destinada à gestão de mercados financeiros. Foram implementadas soluções em *PL/pgSQL* recorrendo a funções, procedimentos armazenados, gatilhos e vistas atualizáveis, garantindo a coerência dos dados e o cumprimento das regras de negócio definidas no enunciado.

As restrições implementadas incluem a validação de números de identificação fiscal e a prevenção de contactos duplicados para um mesmo cliente. Foram igualmente desenvolvidas funções para cálculo de médias móveis e obtenção de informação detalhada de portefólios. Adicionalmente, foi criado um procedimento responsável pela atualização dos valores diários dos instrumentos financeiros e uma vista atualizável para simplificar a gestão de contactos de clientes.

As decisões tomadas privilegiaram reutilização de objectos existentes, encapsulamento da lógica de negócio na base de dados e manutenção da consistência dos dados.

Abstract

This project focuses on the development of integrity mechanisms and business logic in a relational database designed for financial market management. Solutions were implemented in *PL/pgSQL* using functions, stored procedures, triggers and updatable views, ensuring data consistency and compliance with the business rules defined in the assignment.

The implemented constraints include validation of tax identification numbers and prevention of duplicate contacts for the same client. Functions were also developed to calculate moving averages and retrieve detailed portfolio information. Additionally, a stored procedure responsible for updating daily financial instrument values and an updatable view to simplify customer contact management were created.

The adopted decisions prioritized reuse of existing database objects, encapsulation of business logic within the database layer, and preservation of data consistency.

Listagens

Listagem 1 – Comando para executar a aplicação	8
--	---

Índice

1. Introdução	1
2. Implementação da Base de Dados	2
2.1 Restrições de Integridade	2
2.2 Função de Média Móvel	2
2.3 Função de Listagem de Portefólio	3
2.4 Procedimento de Atualização de Valores Diários	3
2.5 Vista Atualizável de Contactos	3
3. Implementação da Camada Aplicacional	5
3.1 Arquitetura	5
3.2 Persistência com JPA	5
3.3 Validações na Aplicação	6
3.4 Integração com a Vista Atualizável	6
3.5 Reutilização da Lógica da Base de Dados	7
3.6 <i>Optimistic Locking</i>	7
3.7 Execução da Aplicação	8
4. Conclusão	9
5. Referências	10

1. Introdução

O presente trabalho teve como objetivo desenvolver um sistema de gestão de mercados financeiros recorrendo a *PostgreSQL* e *Jakarta Persistence (JPA)*. A implementação procurou garantir simultaneamente a integridade dos dados ao nível da base de dados e uma camada aplicacional responsável pela interação com o utilizador e gestão das operações de negócio.

Ao longo do desenvolvimento foi dada prioridade à reutilização dos objetos disponibilizados pelo enunciado e à separação de responsabilidades entre a base de dados e a aplicação. Desta forma, as regras críticas de integridade foram implementadas na base de dados, garantindo a sua aplicação independentemente do cliente utilizado, enquanto as validações relacionadas com a experiência de utilização foram efetuadas na aplicação para fornecer feedback imediato ao utilizador.

2. Implementação da Base de Dados

2.1 Restrições de Integridade

A validação do *NIF* e a prevenção de contactos duplicados foram implementadas através de gatilhos *PL/pgSQL*. A principal decisão consistiu em garantir estas regras ao nível da base de dados, em vez de depender exclusivamente da aplicação.

Embora a aplicação realize validações preliminares antes de submeter os dados, estas verificações têm apenas como objetivo melhorar a experiência de utilização, fornecendo *feedback* imediato ao utilizador. A garantia efetiva da integridade dos dados é assegurada pelos gatilhos, uma vez que a base de dados pode ser acedida por outros clientes ou aplicações que não utilizem a interface desenvolvida neste projeto.

Optou-se pela utilização de gatilhos *BEFORE INSERT* e *BEFORE UPDATE*, permitindo que as validações sejam executadas antes da persistência dos dados. Desta forma, operações inválidas são imediatamente rejeitadas e nunca chegam a ser escritas na base de dados.

No caso da validação do *NIF*, o gatilho verifica se o valor respeita as regras definidas para este identificador antes de permitir a inserção ou atualização do registo. Relativamente aos contactos, foi implementada uma validação que impede a existência de contactos repetidos para o mesmo cliente.

A utilização de gatilhos permite ainda reutilizar a mesma lógica independentemente da origem da operação, assegurando que todas as inserções e atualizações respeitam as mesmas regras de integridade.

2.2 Função de Média Móvel

A função foi implementada na base de dados por se tratar de uma agregação executada sobre um conjunto potencialmente elevado de registos históricos.

Uma alternativa seria obter todos os registos necessários na aplicação e realizar aí o cálculo. No entanto, essa abordagem implicaria a transferência de um volume significativamente superior de dados e duplicaria lógica que poderia vir a ser utilizada por outros consumidores da base de dados.

Ao implementar o cálculo em *SQL*, a operação é executada junto dos dados, reduzindo o tráfego entre aplicação e servidor e beneficiando dos mecanismos de otimização disponibilizados pelo *PostgreSQL*. Esta abordagem promove igualmente a reutilização da funcionalidade por qualquer aplicação que necessite da mesma informação.

2.3 Função de Listagem de Portefólio

A função de listagem do portefólio agrega informação proveniente de várias tabelas e calcula indicadores financeiros necessários à apresentação das posições de um cliente.

Optou-se por realizar estes cálculos na base de dados em vez da aplicação, uma vez que a informação necessária já se encontra armazenada no servidor. Desta forma, evita-se a transferência de dados intermédios e garante-se que todos os consumidores da informação obtêm resultados consistentes.

Esta abordagem reduz também a duplicação de lógica de negócio, uma vez que os cálculos financeiros ficam centralizados numa única implementação.

2.4 Procedimento de Atualização de Valores Diários

A atualização dos valores diários dos instrumentos foi implementada através do procedimento armazenado *p_atualizaValorDiario*.

O procedimento garante a manutenção dos valores mínimo, máximo e fecho ao longo do dia, preservando simultaneamente o valor de abertura correspondente à primeira cotação registada.

Em vez de distribuir esta lógica pela aplicação, optou-se por centralizá-la num procedimento armazenado. Desta forma, todas as atualizações seguem exatamente o mesmo processo, independentemente da origem da operação.

Adicionalmente, a execução da lógica na base de dados permite que todas as alterações ocorram no mesmo contexto transacional, reduzindo o risco de inconsistências e simplificando a camada aplicacional. A aplicação limita-se a invocar o procedimento, delegando na base de dados a responsabilidade pela execução das regras de negócio associadas.

2.5 Vista Atualizável de Contactos

A vista *contacto_cliente* foi criada para disponibilizar uma representação única dos contactos dos clientes, abstraindo a existência de diferentes tipos de contacto (telefone e email). Desta forma, a aplicação consegue trabalhar sobre uma única estrutura lógica sem necessitar de conhecer os detalhes da implementação física da base de dados.

No entanto, por resultar da combinação de múltiplas tabelas, a vista não suporta diretamente operações de escrita. Para permitir a criação de novos contactos através da própria vista, foi implementado um gatilho *INSTEAD OF INSERT*, responsável por encaminhar os dados para as tabelas adequadas.

A principal vantagem desta abordagem é a centralização da lógica de persistência na base de dados. A aplicação apenas necessita de efetuar operações de inserção sobre a vista, ficando a responsabilidade de encaminhar os dados para as tabelas cliente, *contacto_email* e *contacto_telefone* a cargo do gatilho. Isto reduz o acoplamento entre a aplicação e o modelo relacional, permitindo alterações futuras à estrutura interna sem impacto direto na camada aplicacional.

Para além do gatilho associado à vista, foram ainda implementados gatilhos de validação recorrendo aos eventos *BEFORE INSERT* e *BEFORE UPDATE*.

A escolha de gatilhos *BEFORE* deve-se ao facto de as validações terem de ocorrer antes da escrita efetiva dos dados. Caso a validação fosse realizada através de um gatilho *AFTER*, a operação de inserção ou atualização já teria sido executada, obrigando posteriormente à reversão da transação em caso de erro.

Por exemplo, na validação do *NIF*, o gatilho verifica a validade do número antes de permitir a inserção do registo. Se o valor for inválido, é gerada uma exceção e a operação é imediatamente cancelada.

Da mesma forma, a verificação de contactos duplicados é efetuada antes da persistência dos dados. Esta abordagem evita que estados inválidos sejam temporariamente introduzidos na base de dados, mesmo durante o processamento da transação.

A utilização de gatilhos *BEFORE* permite ainda reutilizar a mesma lógica tanto para inserções como para atualizações, garantindo que as regras de integridade são sempre aplicadas independentemente da origem da operação. Assim, mesmo que os dados sejam inseridos por aplicações externas ou diretamente através de *SQL*, as restrições continuam a ser respeitadas.

Esta decisão permitiu que a aplicação persistisse contactos através de uma única entidade *JPA* associada à vista, sem necessidade de distinguir explicitamente entre contactos telefónicos e contactos de correio eletrónico.

3. Implementação da Camada Aplicacional

3.1 Arquitetura

A aplicação foi desenvolvida segundo uma arquitetura em camadas composta por interface de utilizador, serviços, repositórios e entidades *JPA*. Esta organização teve como principal objetivo separar responsabilidades e reduzir o acoplamento entre os diferentes componentes do sistema.

A interface de utilizador é responsável pela interação com o utilizador, recolha de dados e validações preliminares. Os serviços concentram a lógica de negócio e coordenam operações que envolvem múltiplas entidades ou mecanismos de persistência. Os repositórios encapsulam o acesso aos dados e as entidades representam o modelo persistente da aplicação.

Esta separação permitiu que alterações na camada de persistência tivessem um impacto reduzido nas restantes componentes do sistema. Adicionalmente, a lógica de negócio ficou concentrada nos serviços, evitando a sua dispersão pela interface de utilizador ou pelas classes responsáveis pelo acesso aos dados.

3.2 Persistência com *JPA*

A persistência foi implementada recorrendo a *Jakarta Persistence (JPA)*, permitindo mapear as tabelas da base de dados para entidades *Java*.

A utilização de *JPA* permitiu trabalhar diretamente sobre objetos do domínio em vez de construir instruções *SQL* para as operações mais frequentes. Esta abordagem reduz a quantidade de código dependente do sistema de gestão de bases de dados e facilita a manutenção da aplicação.

As entidades foram modeladas de forma a refletir a estrutura relacional da base de dados, recorrendo às anotações disponibilizadas pelo *JPA* para representar chaves primárias, associações e relacionamentos entre entidades.

Optou-se por utilizar o mecanismo de gestão de entidades disponibilizado pelo *JPA* para as operações *CRUD* mais comuns, reservando a utilização de *SQL* e *PL/pgSQL* para funcionalidades cuja execução na base de dados apresentava vantagens significativas ao nível da integridade, desempenho ou reutilização.

Esta decisão permitiu combinar as vantagens da programação orientada a objetos com as funcionalidades disponibilizadas pelo *PostgreSQL*, utilizando cada tecnologia nos cenários para os quais é mais adequada.

3.3 Validações na Aplicação

Embora a base de dados implemente mecanismos próprios de validação e integridade, foram introduzidas validações adicionais na aplicação.

Durante a criação e atualização de clientes são verificados formatos de *NIF*, cartão de cidadão, endereço de correio eletrónico, número de telefone e códigos *ISIN*. Estas validações são executadas antes da realização de qualquer operação de persistência.

A principal motivação para esta decisão foi melhorar a experiência de utilização. Em vez de permitir que uma operação percorra todo o processo de persistência para apenas falhar posteriormente na base de dados, a aplicação consegue identificar erros simples imediatamente após a introdução dos dados.

Contudo, estas validações não substituem as restrições existentes na base de dados. A aplicação assume apenas uma função de apoio ao utilizador, enquanto a garantia efetiva da integridade dos dados permanece centralizada na camada de persistência.

Esta abordagem segue o princípio de defesa em profundidade, em que diferentes camadas do sistema colaboram para garantir a qualidade dos dados sem depender exclusivamente de um único mecanismo de validação.

3.4 Integração com a Vista Atualizável

A criação de clientes foi implementada recorrendo à vista atualizável desenvolvida na base de dados.

Em vez de inserir diretamente dados nas tabelas associadas aos clientes, contactos telefónicos e contactos de correio eletrónico, a aplicação interage com uma única entidade que representa a vista *contacto_cliente*.

Esta decisão permitiu simplificar significativamente a lógica da aplicação. A camada aplicacional deixa de necessitar de conhecer os detalhes da estrutura física da base de dados, delegando essa responsabilidade para os gatilhos *INSTEAD OF* implementados na própria vista.

Desta forma, a aplicação trabalha sobre um modelo mais próximo dos conceitos do domínio do problema, enquanto a base de dados se encarrega da distribuição dos dados pelas tabelas adequadas.

Para além de reduzir a complexidade da aplicação, esta solução promove uma maior independência entre as camadas do sistema. Alterações futuras ao modelo relacional podem ser absorvidas pela vista e respetivos gatilhos sem necessidade de modificar a lógica da aplicação.

3.5 Reutilização da Lógica da Base de Dados

Sempre que possível, a aplicação reutiliza a lógica já existente na base de dados em vez de a reimplementar em *Java*.

Um exemplo desta abordagem é a atualização dos valores diários dos instrumentos financeiros. Em vez de recalcular mínimos, máximos, valores de abertura e fecho na aplicação, esta limita-se a invocar o procedimento armazenado desenvolvido para esse efeito.

A principal vantagem desta decisão é evitar duplicação de lógica de negócio. Caso os mesmos cálculos fossem implementados tanto na base de dados como na aplicação, existiria o risco de divergências entre implementações e um aumento da complexidade de manutenção.

Ao centralizar estas regras na base de dados, garante-se que qualquer aplicação que utilize o sistema beneficia exatamente da mesma lógica de negócio e produz resultados consistentes.

Esta abordagem permitiu igualmente simplificar a camada aplicacional, que passa a assumir um papel de coordenação das operações em vez de replicar funcionalidades já disponibilizadas pelo servidor de base de dados.

3.6 *Optimistic Locking*

A atualização concorrente dos dados dos clientes foi implementada recorrendo ao mecanismo de *optimistic locking* disponibilizado pelo *JPA*.

Para suportar este mecanismo foi adicionada uma coluna de versão à entidade Cliente, mapeada através da anotação *@Version*. Sempre que um registo é alterado, o valor desta coluna é automaticamente incrementado pelo mecanismo de persistência.

Durante uma operação de atualização, a aplicação obtém a entidade utilizando bloqueio otimista. No momento da escrita, o *JPA* verifica se a versão atualmente existente na base de dados corresponde à versão originalmente lida. Caso outro utilizador tenha alterado o mesmo registo entretanto, é gerada uma exceção do tipo *OptimisticLockException*.

A escolha deste mecanismo deve-se às características do sistema desenvolvido. Em cenários deste tipo, os conflitos concorrentes tendem a ser relativamente pouco frequentes, tornando desnecessária a utilização de bloqueios pessimistas que impediriam outros utilizadores de aceder aos mesmos registos durante toda a duração da operação.

O *optimistic locking* permite assim maximizar a concorrência e reduzir o tempo de retenção de recursos, detetando conflitos apenas quando estes efetivamente ocorrem.

Nos testes realizados foram simuladas duas sessões independentes a alterar os dados do mesmo cliente. Quando a primeira sessão concluiu a atualização, a versão da entidade foi incrementada. Posteriormente, a tentativa de atualização efetuada pela segunda sessão originou uma *OptimisticLockException*, confirmando o correto funcionamento do mecanismo de controlo de concorrência implementado.

Para suportar o mecanismo de optimistic locking foi necessário introduzir uma coluna de versão nas entidades sujeitas a controlo de concorrência.

3.7 Execução da Aplicação

Para compilar, testar e executar a aplicação foi utilizado o sistema de gestão de dependências *Maven*. A execução completa do projeto pode ser realizada através do seguinte comando:

```
mvn clean package exec:java
```

Listagem 1 – Comando para executar a aplicação

Este comando remove artefactos de compilações anteriores (*clean*), compila o projeto e executa os testes automáticos (*package*). Caso todos os testes sejam concluídos com sucesso, a aplicação é posteriormente iniciada através do objetivo *exec:java*.

Desta forma, garante-se que a aplicação apenas é executada após a validação do correto funcionamento dos componentes testados.

4. Conclusões

O desenvolvimento deste projeto permitiu aplicar conceitos fundamentais de bases de dados, persistência de informação e desenvolvimento de aplicações empresariais, recorrendo à integração entre *PostgreSQL* e *Jakarta Persistence* (*JPA*).

Ao longo da implementação procurou-se atribuir cada responsabilidade à camada mais adequada. As regras de integridade e a lógica diretamente relacionada com os dados foram centralizadas na base de dados através de gatilhos, funções, procedimentos armazenados e vistas atualizáveis. Esta decisão permitiu garantir que as regras de negócio são aplicadas de forma consistente independentemente da aplicação que aceda aos dados, reduzindo simultaneamente a duplicação de lógica e o risco de inconsistências.

Por outro lado, a camada aplicacional foi desenvolvida com foco na usabilidade, validação preliminar dos dados e coordenação das operações de negócio. A utilização de uma arquitetura em camadas permitiu separar claramente as responsabilidades da interface de utilizador, dos serviços e dos mecanismos de persistência, facilitando a manutenção e evolução do sistema.

A adoção de *JPA* simplificou a interação com a base de dados através de um modelo orientado a objetos, mantendo simultaneamente a possibilidade de reutilizar funcionalidades implementadas diretamente em *SQL* quando tal apresentava vantagens ao nível do desempenho, reutilização ou integridade dos dados. Esta combinação permitiu tirar partido dos pontos fortes de ambas as tecnologias sem concentrar toda a lógica numa única camada.

Um dos aspetos mais relevantes do projeto foi a preocupação com a consistência dos dados em cenários concorrentes. A implementação do mecanismo de optimistic locking demonstrou a importância do controlo de concorrência em sistemas multiutilizador, permitindo detetar conflitos de atualização sem recorrer a bloqueios permanentes dos registos.

Em termos globais, considera-se que os objetivos definidos no enunciado foram cumpridos. O sistema desenvolvido suporta a gestão de clientes, contactos, portefólios e instrumentos financeiros, garantindo simultaneamente a integridade da informação, a reutilização da lógica de negócio e uma separação adequada entre as diferentes camadas da aplicação.

Por fim, o projeto permitiu consolidar conhecimentos relacionados com modelação de dados, programação em *SQL* e *PL/pgSQL*, utilização de mecanismos de persistência orientados a objetos e integração entre base de dados e aplicação, evidenciando a importância de uma correta distribuição de responsabilidades entre estas duas componentes de um sistema de informação.

5. Referências

- [1] PostgreSQL Global Development Group. PostgreSQL Documentation – PL/pgSQL. Disponível em: <https://www.postgresql.org/docs/>
- [2] PostgreSQL Global Development Group. PostgreSQL Documentation – Triggers. Disponível em: <https://www.postgresql.org/docs/current/plpgsql-trigger.html>
- [3] PostgreSQL Global Development Group. PostgreSQL Documentation – Views. Disponível em: <https://www.postgresql.org/docs/current/sql-createview.html>
- [4] PostgreSQL Global Development Group. PostgreSQL Documentation – Stored Procedures. Disponível em: <https://www.postgresql.org/docs/current/sql-createprocedure.html>
- [5] Jakarta EE. Jakarta Persistence Specification. Disponível em: <https://jakarta.ee/specifications/persistence/>
- [6] Vlad Mihalcea. High-Performance Java Persistence. Amazon, 2020.
- [7] Martin Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- [8] Instituto Superior de Engenharia de Lisboa. Sistemas de Informação – Projecto 2025-2026.
- [9] Craig Walls. Spring in Action. Manning Publications, 2022.
- [10] Gavin King, Christian Bauer. Hibernate in Action. Manning Publications, 2004.
- [11] Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
- [12] Oracle. Java Persistence API (JPA) Overview. Disponível em: <https://docs.oracle.com/javaee/7/tutorial/persistence-intro.htm>
- [13] Hibernate ORM Documentation. Optimistic Locking. Disponível em: https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html
- [14] Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [15] Elmasri, Ramez; Navathe, Shamkant. Fundamentals of Database Systems. 7th Edition. Pearson, 2015.